



UNIT INFORMATION	
COURSE – (LEADING TO)	B.Sc (Hons) Software Engineering
UNIT NAME	Business Component Development I
UNIT ID	JIAT/BCD I
ASSIGNMENT ID	JIAT/BCD I/EX/01

STUDENT INFORMATION	
STUDENT NAME	Mohamed Fareed Abdullah
STUDENT ID	200506301171
BATCH	Cohort 8 - BCU
BRANCH	Kandy

Table of Contents

Table of Contents.....	2
ER Diagram	3
1. Technical Implementation Documentation.....	4
1.1 Project Overview and Business Context.....	4
1.2 Critical Analysis of Java EE Platform Selection.....	4
1.3 System Architecture.....	4
1.4 Session Bean Architecture and Lifecycle Optimisation.....	5
1.5 Exception Handling and Business Rule Enforcement	5
1.6 JNDI, Dependency Injection and Resource Configuration.....	6
1.7 Asynchronous Processing and JMS Messaging Architecture.....	6
1.8 Message-Driven Bean Design.....	7
1.9 Database Integration and Connection Pool Optimisation	7
1.10 Non-Functional Requirements Strategy: Scalability and Reliability	7
1.11 Deployment Architecture on Payara.....	8
1.12 Performance Monitoring Implementation.....	8
References.....	9

ER Diagram

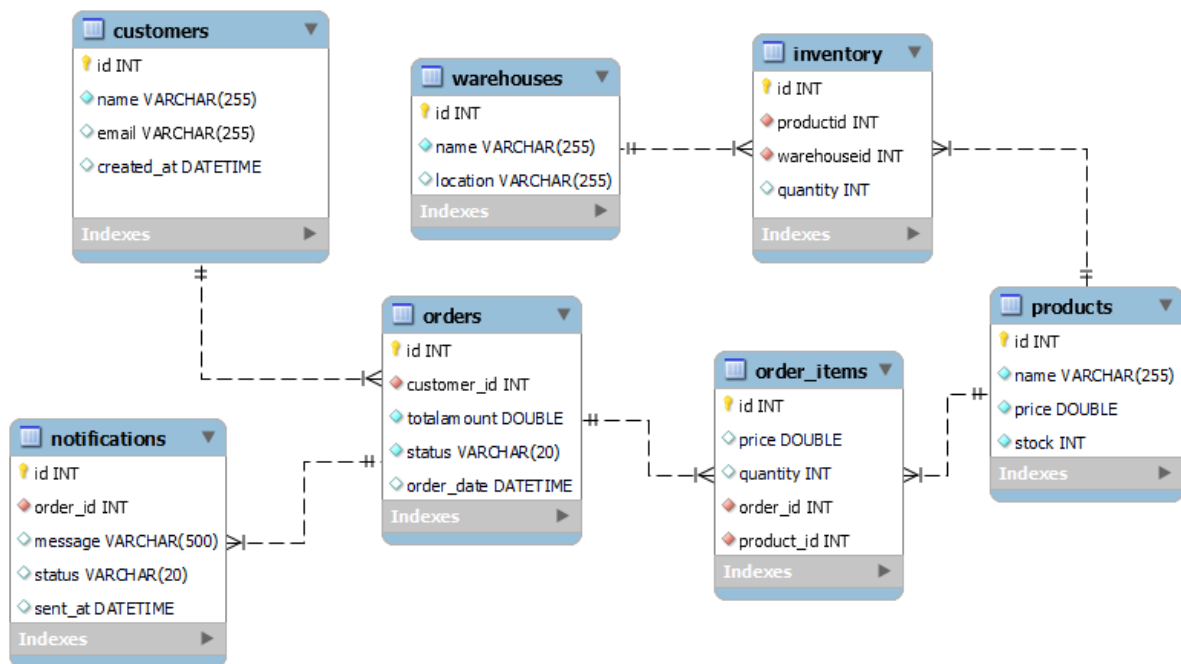


Figure 1: ER Diagram

The diagram above represents the relational schema realised through the JPA entity model in *techmart-ejb/src/main/java/lk/techmart/model*. **Product** is the catalogue aggregate root and relates to **Inventory** in a one-to-many relationship, with each **Inventory** row further scoped to a specific **Warehouse**, allowing the same product's stock to be tracked independently across the Colombo, Kandy and Galle facilities. **Customer** relates one-to-many to **Order**, and each **Order** owns a cascading collection of **OrderItem** rows that reference **Product**. **Order** additionally relates one-to-many to **Notification**, which records the outcome of the asynchronous confirmation workflow triggered after checkout. Together these relationships let the system reconstruct, for any order, exactly which warehouses fulfilled it and what communication was sent to the customer.

1. Technical Implementation Documentation

1.1 Project Overview and Business Context

TechMart Online models a retailer's transition from a legacy monolithic point-of-sale system to a Jakarta EE 10 architecture supporting 10,000-plus concurrent users with sub-second response times. The prototype (package `lk.techmart`) is a multi-module Maven build (`techmart-ejb`, `techmart-war`, `techmart-ear`) packaged as one Enterprise Archive, deployed to Payara and backed by MySQL 8. It supports catalogue browsing, cart management, order placement, asynchronous stock reconciliation across three warehouses, customer notification and live performance metrics, directly addressing the brief's real-time inventory, automated processing and scalable session-management requirements.

1.2 Critical Analysis of Java EE Platform Selection

Jakarta EE was chosen over Spring Boot or a Node.js/.NET microservices split because session beans, JPA, JMS and CDI are standardised, specification-driven APIs rather than one vendor's framework, so `techmart-ejb` can redeploy on GlassFish, Payara, WildFly or TomEE unchanged, avoiding vendor lock-in (Eclipse Foundation, 2022a). Container-managed transactions and pooling, visible in the `@Stateless/@Singleton/@MessageDriven` beans, remove a class of concurrency bugs a hand-rolled Spring or Node.js service would otherwise manage explicitly. The trade-off is velocity: Spring Boot offers a comparable component model with a larger ecosystem, faster startup and no application-server requirement, and a Node.js/Go microservices split would scale more easily in Kubernetes. For TechMart, the brief mandates Jakarta EE, and the single cohesive order-processing domain suits its integrated model; the main residual limitation is that scaling beyond one Payara instance needs explicit clustering, which the current single-node deployment does not yet provide.

1.3 System Architecture

The application follows a three-tier layering (Fowler, 2002): the presentation tier (`techmart-war`) has five Servlets and matching JSP/JSTL views with no business logic in the view layer; the business tier (`techmart-ejb`) hosts four stateless beans, one stateful bean, one singleton and one message-driven bean, packaged separately from the web tier; the persistence tier uses JPA via EclipseLink against MySQL through a JTA-managed pool. Both modules combine into one Jakarta EE 10 EAR (Eclipse Foundation, 2022a). A cross-cutting interceptor (`PerformanceInterCeptor`) times business methods and feeds `MetricsBean`, giving the application built-in performance observability. Because the EAR currently targets a single Payara instance, horizontal scaling toward the 10,000-user target would require an

external load balancer distributing requests across cloned Payara nodes, with session affinity or a distributed session store to keep ShoppingCartBean instances consistent across nodes — a clustering layer this prototype deliberately leaves for a later iteration rather than the single-node architecture demonstrated here.

1.4 Session Bean Architecture and Lifecycle Optimisation

All three EJB session bean types are used deliberately. ProductServiceBean and CustomerServiceBean are @Stateless, performing idempotent reads the container can freely pool, the correct choice for high-throughput read access. OrderServiceBean is also stateless, validating stock and persisting orders with no state carried between calls. ShoppingCartBean is the sole @Stateful bean, needed because a cart must persist across requests; it is also, redundantly, annotated with CDI's @SessionScoped, mixing EJB and CDI scope management, and at 10,000 concurrent users a pure CDI-scoped bean would avoid the extra passivation overhead stateful EJBs incur per client. MetricsBean is a @Singleton with container-managed concurrency and a NOT_SUPPORTED transaction attribute, correct for in-memory counters, though addProductFetchTime's separate REQUIRES_NEW annotation is redundant alongside its own synchronized keyword.

None of the four bean types currently implements a lifecycle callback. This is a genuine optimisation gap rather than a neutral omission: ShoppingCartBean has no @Remove-annotated checkout method and no tuned @StatefulTimeout, so the container can only reclaim a cart's memory at session expiry rather than immediately after checkout, directly inflating heap residency under sustained load. A @PostConstruct initialiser on MetricsBean would also let the singleton log its own activation deterministically rather than relying on implicit lazy instantiation, and a @PreDestroy hook would allow any future persistent metrics snapshot to flush cleanly on shutdown. Adding these callbacks (Eclipse Foundation, 2024) would tie bean lifecycle explicitly to business events — checkout completion, application shutdown — rather than leaving it entirely to container defaults.

1.5 Exception Handling and Business Rule Enforcement

Stock sufficiency is enforced via a custom unchecked exception, InsufficientStockException, carrying the product name and requested/available quantities. OrderServiceBean.createOrder() checks aggregate warehouse stock before persistence and throws it if insufficient, causing the container to roll back any changes made within that transactional context. PlaceOrderServlet catches it separately from generic exceptions, restores the cart, and redirects with actionable error details. The check-then-act window

between this validation and the actual deduction in `OrderProcessingMDB` remains open, since the two operations run in separate transactions on separate threads.

1.6 JNDI, Dependency Injection and Resource Configuration

Dependency injection (`@EJB`, `@Inject`, `@Resource`) is used throughout rather than manual JNDI lookups, reducing boilerplate and container coupling (Burke and Monson-Haefel, 2006). The trade-off is portability: the JNDI names driving the application — `jdbc/TechMartDataSource`, configured as the JTA data source in `persistence.xml`, and `jms/TechmartConnectionFactory` and `jms/TechmartOrderQueue`, injected via `@Resource` in `PlaceOrderServlet` — are hard-coded rather than externalised as resource-ref overrides, so retargeting the application to a different environment requires a code or configuration-file change rather than a deployment-time mapping. This was a real issue encountered during Payara deployment, where the bound resource names initially did not match these hard-coded lookups.

1.7 Asynchronous Processing and JMS Messaging Architecture

Order acceptance and fulfilment are deliberately split. `PlaceOrderServlet` persists the order synchronously via `OrderServiceBean`, then sends a point-to-point message to `jms/TechmartOrderQueue`, consistent with the Jakarta Messaging specification (Eclipse Foundation, 2022b), so the customer-facing request avoids the cost of warehouse deduction and notification. Separately, `NotificationServiceBean.sendOrderConfirmationEmail()` is `@Asynchronous`, returning an unused `Future` so the three-second simulated mail delay never blocks checkout. The two mechanisms suit different reliability needs: JMS persists messages on the broker even if the message-driven bean is briefly unavailable, while the `@Asynchronous Future` is lost on a container restart, so routing the business-critical stock deduction through JMS is the correct choice.

Only the point-to-point pattern is implemented; no publish/subscribe destination exists. This is a defensible scope decision rather than an oversight: order confirmation has exactly one intended consumer (`OrderProcessingMDB`), so a `Topic`'s fan-out semantics would add configuration overhead without a second subscriber to justify it. Publish/subscribe would earn its complexity if TechMart needed to broadcast a single event to multiple independent consumers — for example, a stock-level-changed event consumed simultaneously by a warehouse dashboard, a re-order-trigger service and a customer back-in-stock notifier — none of which the current three-warehouse prototype requires, but all of which are plausible extensions once inventory synchronisation moves beyond a single MDB.

1.8 Message-Driven Bean Design

OrderProcessingMDB deducts stock proportionally across every warehouse holding an ordered product, rounding each share and absorbing any remainder into the largest-stock warehouse so the total deducted matches exactly (Eclipse Foundation, 2024). Two gaps remain: Inventory rows are read and updated with no pessimistic lock, so concurrent messages for the same product can oversell stock, precisely what the brief asks the architecture to prevent; and the catch block only logs failures, leaving orders PENDING with no retry or dead-letter configuration, so a poison message silently strands an order. Throughput also depends entirely on the server's default MDB pool size, since no maxSession tuning is set; increasing it would raise throughput at the cost of more concurrent writes worsening the locking gap.

1.9 Database Integration and Connection Pool Optimisation

persistence.xml configures a JTA pool sized 200 to 1,000 connections and a shared second-level cache for Product, matching the 10,000-user target and reducing catalogue round-trips. database-seed.sql adds indexes on the foreign keys used by the hot queries across order items, inventory, orders and notifications. schema-generation.database.action is set to none, so schema and index ownership sits entirely in the seed script rather than JPA auto-DDL, a deliberate, version-controlled choice.

1.10 Non-Functional Requirements Strategy: Scalability and Reliability

Beyond the component-level choices above, two NFRs need an explicit mitigation strategy at the architecture level. Scalability currently relies on a large connection pool and a stateless-heavy bean design, both of which scale vertically within one Payara instance; genuine horizontal scale-out — the kind the 10,000-user target ultimately needs — would require the load-balanced, multi-node clustering described in Section 1.3, plus an auto-scaling policy that adds Payara nodes when queue depth on TechmartOrderQueue or average request latency crosses a threshold, rather than the fixed single-node capacity currently deployed. Reliability is addressed only partially: JMS decouples order acceptance from fulfilment, but there is no circuit breaker around the one external dependency this architecture would gain in production — a real payment gateway or shipping-carrier API — so a slow or failing third-party call would currently block the calling thread indefinitely rather than failing fast and falling back to a queued retry (Nygard, 2007). Disaster recovery is not yet addressed at all: MySQL runs as a single instance with no read replica or automated backup schedule described in the current configuration, so a database failure would mean full data loss rather than failover to a standby. Each of these is a design gap rather than an implementation bug, and each maps to a concrete next step: introduce a load balancer and cluster configuration, wrap any future external API calls in a circuit breaker, and add MySQL replication with a documented backup and restore procedure.

1.11 Deployment Architecture on Payara

techmart-ear bundles techmart-ejb.jar and techmart-war.war into one Enterprise Archive at context root /techmart. Payara deployment requires three pre-configured JNDI resources — the datasource pool and the two JMS resources described in Section 1.6 (Payara Services Ltd, 2024) — and because pool sizing lives in persistence.xml rather than Payara's own configuration, the two must be kept manually in agreement, a real drift risk confirmed during deployment.

1.12 Performance Monitoring Implementation

Rather than an external APM tool, PerformanceInterCeptor times business methods and MetricsBean aggregates orders, processed messages, average MDB time and catalogue fetch time, surfaced live via metrics.jsp, metricsBar.jsp and the transactions dashboard — low-overhead, dependency-free instrumentation built into the architecture itself.

References

Burke, R. and Monson-Haefel, R. (2006) Mastering Enterprise JavaBeans. 3rd edn. Indianapolis: Wiley.

Eclipse Foundation (2022a) Jakarta EE Platform, Version 10. Available at: <https://jakarta.ee/specifications/platform/10/> (Accessed: 3 July 2026).

Eclipse Foundation (2022b) Jakarta Messaging Specification, Version 3.1. Available at: <https://jakarta.ee/specifications/messaging/3.1/> (Accessed: 3 July 2026).

Eclipse Foundation (2024) Jakarta Enterprise Beans Specification, Version 4.0. Available at: <https://jakarta.ee/specifications/enterprise-beans/4.0/> (Accessed: 3 July 2026).

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Nygard, M.T. (2007) Release It!: Design and Deploy Production-Ready Software. Raleigh: Pragmatic Bookshelf.

Payara Services Ltd (2024) Payara Server Documentation. Available at: <https://docs.payara.fish/> (Accessed: 3 July 2026).